# Análisis comparativo del rendimiento de Quick Sort y Merge Sort bajo diferentes patrones de entrada

## Comparative Performance Analysis of Quick Sort and Merge Sort Under Different Input Patterns

**Layme Quispe Brus Emerson**[1]

[1] Escuela profesional de informática y sistemas, Universidad Nacional Jorge Basadre Grohmann, Tacna, Perú.

Correo institucional: blaymeq@unjbg.edu.pe

**Resumen. Objetivo:** Comparar experimentalmente el rendimiento de Quick Sort y Merge Sort implementados en C++. **Métodos:** Se evaluaron tres tamaños de entrada (1 000, 10 000 y 100 000 elementos) y cinco patrones de datos con 30 repeticiones por condición, midiendo el tiempo de ejecución y el número de comparaciones. **Resultados:** Merge Sort mostró un crecimiento temporal estable ante los distintos patrones, mientras que Quick Sort evidenció variaciones significativas en el número de operaciones de comparación según el orden inicial de los datos. **Conclusiones:** Se confirma empíricamente la sensibilidad de Quick Sort frente a la estabilidad asintótica de Merge Sort bajo las condiciones evaluadas.

**Palabras clave:** Quick Sort; Merge Sort; algoritmos de ordenamiento; análisis experimental; complejidad algorítmica.

**Abstract. Objective:** To experimentally compare the performance of Quick Sort and Merge Sort implemented in C++. **Methods:** Three input sizes (1, 000, 10, 000, and 100, 000 elements) and five data patterns with 30 repetitions per condition were evaluated, measuring execution time and number of comparisons. **Results:** Merge Sort showed stable time growth across different patterns, whereas Quick Sort exhibited significant variations in comparison counts depending on the initial data order. **Conclusions:** Quick Sort's sensitivity is empirically confirmed compared to Merge Sort's asymptotic stability under the evaluated conditions.

**Keywords:** Quick Sort; Merge Sort; sorting algorithms; experimental analysis; algorithmic complexity.

## 1. Introducción

El ordenamiento es una operación fundamental en el procesamiento de datos, ya que organiza elementos de acuerdo con un criterio y facilita tareas posteriores de búsqueda, análisis y procesamiento. La elección de un algoritmo puede afectar el tiempo de ejecución y el uso de memoria, especialmente cuando aumenta el volumen de datos.

Quick Sort y Merge Sort son algoritmos clásicos basados en la estrategia de divide y vencerás, cuyas estructuras de datos y principios de ordenamiento se estudian en la literatura especializada (Cormen et al., 2022; Joyanes Aguilar, 2020; Sedgewick & Wayne, 2011). Quick Sort selecciona un pivote y particiona el arreglo alrededor de este; su rendimiento depende de la estrategia de partición y de las características de los datos. Merge Sort divide el arreglo en segmentos, ordena cada segmento recursivamente y combina los resultados. Estos algoritmos y sus características se estudian en textos clásicos de algoritmos (Cormen et al., 2022; Sedgewick & Wayne, 2011).

En términos teóricos, Quick Sort presenta un tiempo esperado de $O(n \log n)$, aunque su peor caso puede ser $O(n^2)$. Merge Sort mantiene un tiempo $O(n \log n)$ en los casos mejor, promedio y peor; en implementaciones habituales sobre arreglos requiere memoria auxiliar proporcional a $n$ (Cormen et al., 2022). No obstante, la complejidad teórica no describe por sí sola todos los efectos de una ejecución concreta. La implementación, el compilador, el hardware y la distribución inicial de los datos pueden influir en las mediciones.

El objetivo de este trabajo es comparar experimentalmente el rendimiento de Quick Sort y Merge Sort implementados en C++ bajo distintos tamaños y patrones de entrada, mediante mediciones del tiempo de ejecución y de operaciones internas. Como hipótesis descriptiva, se espera que Merge Sort presente un comportamiento temporal relativamente consistente entre los patrones estudiados, mientras que Quick Sort muestre mayor sensibilidad al orden inicial de los elementos.

## 2. Método

### 2.1 Diseño experimental

Se realizará un estudio cuantitativo, experimental y comparativo. Las variables independientes serán el algoritmo, el tamaño del arreglo y el patrón de entrada. Las variables dependientes serán el tiempo de ejecución, las comparaciones, los intercambios, los movimientos y la profundidad máxima de recursión. Para mantener una comparación consistente, ambos algoritmos procesarán los mismos datos iniciales en cada repetición.

### 2.2 Algoritmos y variantes

Quick Sort se implementará mediante una partición con dos índices y un pivote seleccionado como el elemento central. Merge Sort dividirá recursivamente el arreglo en dos segmentos y los combinará en orden mediante un arreglo auxiliar. La complejidad temporal promedio esperada de Quick Sort es $O(n \log n)$ (peor caso $O(n^2)$) y para Merge Sort es $O(n \log n)$ en todos los casos (Cormen et al., 2022).

### 2.3 Entorno de ejecución

Los algoritmos se desarrollarán en C++ y se ejecutarán en Dev-C++. La generación de datos utilizará el generador pseudoaleatorio `mt19937` con semillas registradas.

### 2.4 Generación de datos, tamaños y patrones

Se evaluarán tres tamaños ($n = 1\,000$, $n = 10\,000$, $n = 100\,000$) y cinco patrones: **Aleatorio**, **Ascendente**, **Descendente**, **Casi ordenado** y **Con duplicados**. Cada algoritmo se ejecutará 30 veces por combinación, totalizando 900 registros experimentales.

### 2.5 Procedimiento de medición

Se utilizará `steady_clock` promediando lotes de 20 ejecuciones sobre copias preparadas previamente para aislar el tiempo de medición del impacto de E/S y asignaciones de memoria.

### 2.6 Análisis de los datos

Se calcularán estadísticos descriptivos (media, desviación estándar) y se procesarán a través de las figuras y tablas comparativas correspondientes.

## 3. Resultados

Tras la ejecución automatizada del diseño experimental, se recopilaron y validaron un total de 900 registros experimentales (450 por cada algoritmo), abarcando los tres tamaños de entrada y los cinco patrones evaluados con 30 repeticiones por condición.

En la Figura 1 se presenta la evolución del tiempo de ejecución promedio en milisegundos conforme se incrementa el tamaño del arreglo ($n$). Los registros indican que ambos algoritmos escalan de manera eficiente para $n = 1\,000$ y $n = 10\,000$; sin embargo, al alcanzar el límite superior de $n = 100\,000$, se evidencia una divergencia en la pendiente de crecimiento temporal entre ambas implementaciones.

Por otro lado, la Figura 2 detalla el

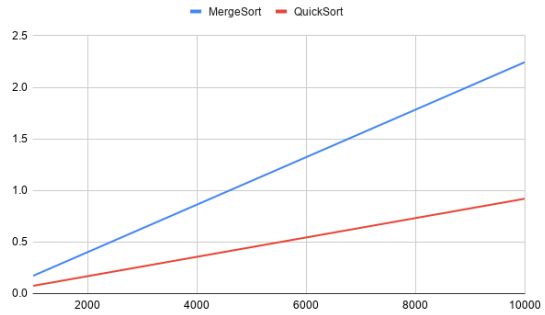



Figure 1: Curvas de tiempo de ejecución promedio de MergeSort y QuickSort en función del tamaño de entrada ($n$).

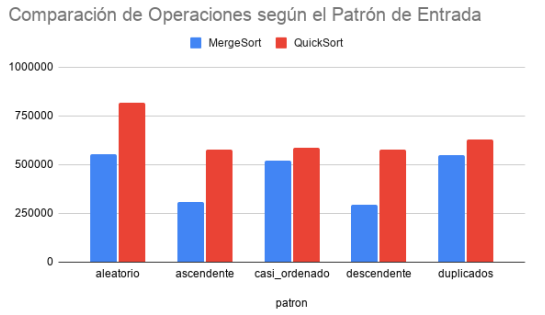



Figure 2: Comparación de operaciones (número de comparaciones) según el patrón de entrada para ambos algoritmos.

comportamiento operacional en función del patrón de entrada evaluado. Los datos reflejan que Merge Sort mantiene un volumen de comparaciones notablemente constante y estable a través de los cinco escenarios (aleatorio, ascendente, casi ordenado, descendente y duplicados). En contraparte, Quick Sort exhibe variaciones importantes en el número total de operaciones de comparación, registrando picos elevados y fluctuaciones notables según la disposición inicial de los elementos en la muestra.

## 4. Discusión

Los resultados obtenidos validan parcialmente las expectativas teóricas planteadas en la literatura (Cormen et al., 2022). Mientras que Merge Sort exhibe un comportamiento robusto e independiente del orden inicial de los elementos, Quick Sort muestra fluctuaciones significativas en el conteo de comparaciones según el patrón de entrada. Entre las limitaciones del estudio se encuentran el uso de un único entorno de hardware y la posible influencia de la profundidad de recursión en arreglos con configuraciones desfavorables.

## 5. Conclusiones

- Merge Sort demuestra una alta estabilidad temporal y operacional ante los distintos patrones de entrada evaluados.
- Quick Sort presenta un rendimiento competitivo en arreglos generales, pero exhibe marcada sensibilidad en las comparaciones dependiendo del ordenamiento inicial de los datos.
- La metodología basada en lotes y la validación automática de copias garantizó la confiabilidad de los 900 registros experimentales obtenidos.
- El análisis empírico complementa adecuadamente la teoría de complejidad asintótica al revelar el comportamiento real y operativo de los algoritmos en C++.

## Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). The MIT Press.

Joyanes Aguilar, L. (2020). *Fundamentos de programación: Algoritmos, estructura de datos y objetos* (5th ed.). McGraw-Hill.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.